# Computer Architecture Lab

## (EL-2012)

## LABORATORY MANUAL

## FALL 2026




# (Lab#07)

## Implementation of Arithmetic Logic Unit (ALU)

Student Name: _____ Roll No: _____ Section: _A__

Date performed: _____, 2026

**MARKS AWARDED: _____ / 10**

**Signature and date of Instructor: _____**

---

**NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD**

Prepared by: Engr. Fasih Ahmad Version: 2.1

Verified by: Dr. Nasir Ali Shah Updated: Fall 2026

## Learning Objectives:

- To design and implement the Arithmetic Logic Unit using Verilog.
- To simulate the ALU using ModelSim.

## Tool Used:

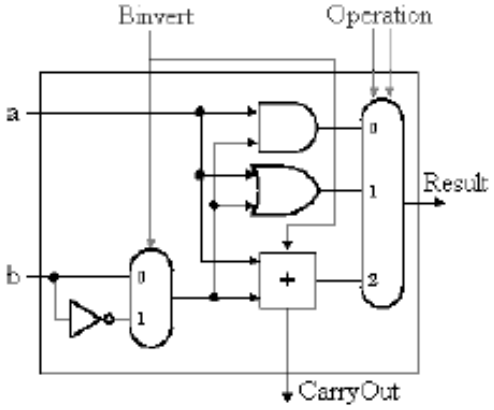
ModelSim

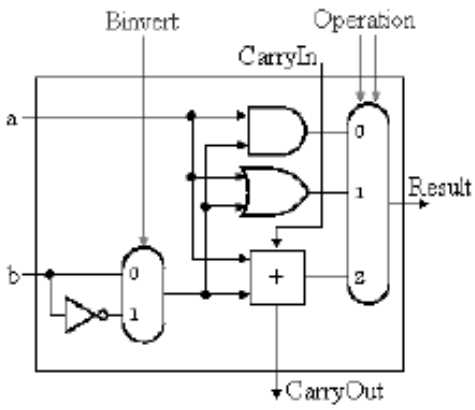
**Arithmetic Logic Unit:**

An arithmetic logic unit (ALU) is a digital circuit used to perform arithmetic and logic operations. It represents the fundamental building block of the central processing unit (CPU) of a computer. Modern CPUs contain very powerful and complex ALUs. In addition to ALUs, modern CPUs contain a control unit (CU).
Most of the operations of a CPU are performed by one or more ALUs, which load data from input registers. A register is a small amount of storage available as part of a CPU. The control unit tells the ALU what operation to perform on that data, and the ALU stores the result in an output register. The control unit moves the data between these registers, the ALU, and memory.

**ALU Design:**

Design of an ALU purely depends upon what type of functionality you want to perform from your ALU. We want to make a 1 bit ALU that will have an AND and OR operations performed by logic gates and ADD and SUB operations. For these operations we would need FULL ADDER, an AND and OR gates and a MUX to decide which operation is to be performed on the inputs. Select line (2bit) of MUX would define which operation is to be performed. The module will have two 1 bit inputs, Carry_in and 2 bit Select line as an Input and 1bit Out and Carry_out as outputs. 1 bit input would be dedicated to decide we want addition or subtraction from ADDER, because both the commands will use the same ADDER. We are already well aware of AND and OR Gates and how they can be utilized in codes to perform logical operations. We can use a FULL ADDER to perform addition of two inputs A and B.



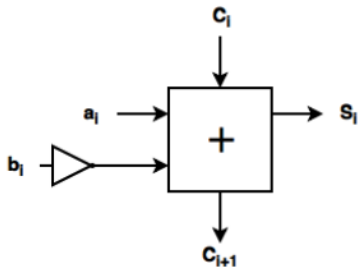



For executing SUB we can use addition command.

To use addition command for SUB we can rewrite the SUB statement as

A-B=A+B'+1

It means we just need to take complement of B and make the carry input as 1 of FULL ADDER to calculate the SUB of two numbers




**Four Bit ALU Design:**

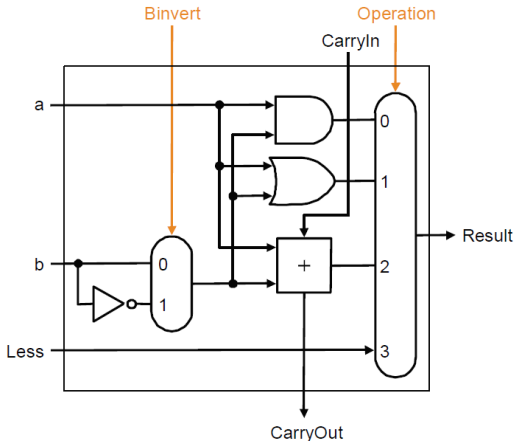
For 4bit ALU, we will have the same design as of 1bit ALU with an extra function, Set Less Than. SLT is used to compare two inputs. After adding the SLT input, we will cascade 4 1bit ALUs to form a 4bit ALU.

**SLT**

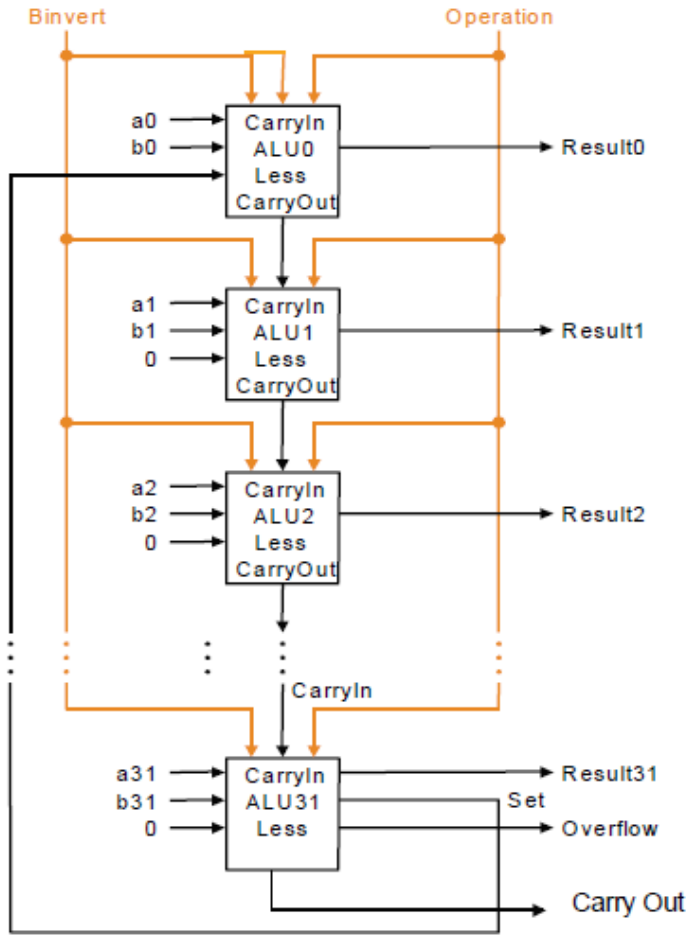
SLT function is defined as:

$$A \text{ slt } B = \begin{cases} 0001 \text{ if } A < B, \text{ i.e. if } A - B < 0 \\ 0000 \text{ if } A > B, \text{ i.e. if } A - B > 0 \end{cases}$$

Thus, each 1-bit ALU should have an additional input (called "Less"), that will provide results for slt function. This input has value 0 for all but 1-bit ALU for the least significant bit. For the least significant bit Less value should be sign of A – B. Now with slt function added, the 1bit ALU design can be represented as:



For 4 or multiple bit ALU, the integration of ALUs for SLT operation can be represented as:




Apart from slt, we will also add some status flags to out ALU which includes Carry Flag, Overflow Flag, Zero Flag and Set Flag. These flag tells about inputs and outputs of operations.

Flags in ALU.

Zero Flag

Determining whether two values are equal requires the ALU to determine whether the result is zero. This can be accomplished by feeding each bit of the result into a NOR gate. The beauty of this is that a single multi-port NOR gate requires less hardware than an entire array of equivalent 2-port gates.

Overflow Flag

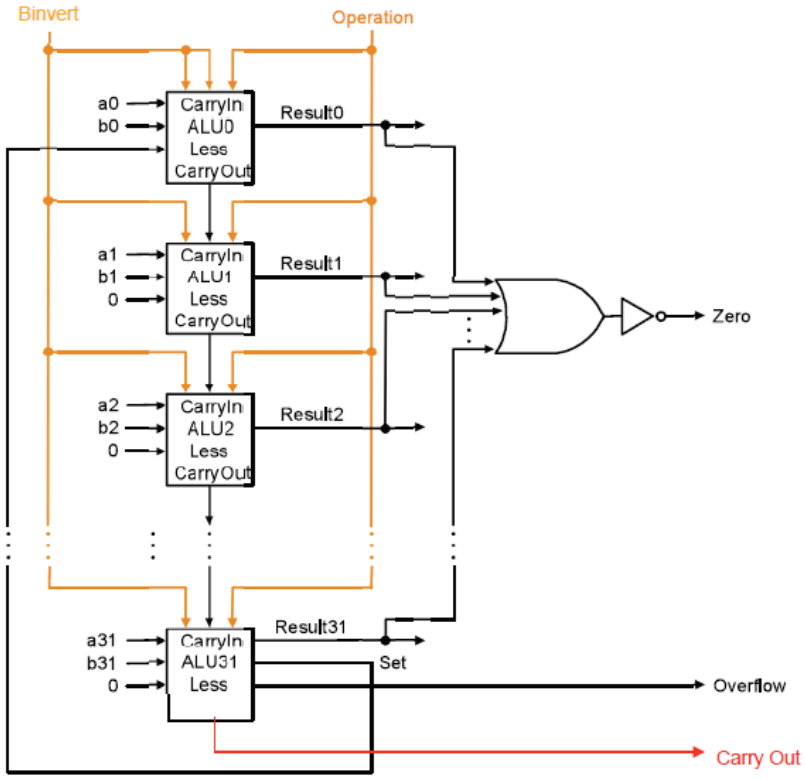
It is good to know when the result of an addition or multiplication is larger than the maximum result size. Likewise, it is also good to know if the result of a subtraction or a division is smaller than possible, and thus creates underflow. Either two separate flags can be used for these

conditions, or one flag can be interpreted in different ways, depending on the input operation.

Carry/Borrow flag

This flag indicates when an operation results in a value larger than the accumulator can represent (carry/overflow) or smaller than the accumulator can represent (borrow/underflow).

The final design on 4bit ALU can be represented as:




Verilog Parameters

Parameters are Verilog constructs that allow a module to be reused with a different specification. For example, a 4-bit adder can be parameterized to accept a value for the number of bits and new parameter values can be passed in during module instantiation. So, an N-bit adder can become a 4-bit, 8-bit or 16-bit adder. They are like arguments to a function that are passed in during a function call.

```
parameter MSB = 7;                  // MSB is a parameter with a constant value 7
parameter REAL = 4.5;               // REAL holds a real number

parameter FIFO_DEPTH = 256,
          MAX_WIDTH = 32;           // Declares two parameters

parameter [7:0] f_const = 2'b3;     // 2 bit value is converted to 8 bits; 8'b3
```

Parameters are basically constants and hence it is illegal to modify their value at runtime. It is illegal to re-declare a name that is already used by a net, variable or another parameter. There are two major types of parameters, module and specify and both accepts a range specification. But they are normally made as wide as the value to be stored requires them to be and hence a range specification is not necessary.
In the new ANSI style of Verilog port declaration, you may declare parameters as shown below.

```
module design_ip
    #(parameter BUS_WIDTH = 32,
      parameter DATA_WIDTH = 64) (

      input [BUS_WIDTH-1:0] addr
      // Other port declarations
);
endmodule
```

Parameters can be overridden with new values during module instantiation. The first part instantiates the module called design_ip by the name d0 where new parameters are passed in within #( ). The second part uses a Verilog construct called defparam to set the new parameter values. The first method is the most used way to pass new parameters in RTL designs. The second method is commonly used in testbench simulations to quickly update the design parameters without having to re-instantiate the module.

```
module tb();

    // Module instantiation override
    design_ip #( .BUS_WIDTH(64), .DATA_WIDTH(128) ) d0 ( /* port list */ );

    // Use of defparam to override parameter
    defparam d0.FIFO_DEPTH = 128;

endmodule
```

Example:
Following is the example of Verilog code for N-Bit Adder.

```
module adder #(parameter WIDTH = 8) (
    input wire  [WIDTH-1:0] a,
    input wire  [WIDTH-1:0] b,
    output wire [WIDTH-1:0] sum,
    output wire carry_out
);
    assign {carry_out, sum} = a + b;  // Concatenation to assign to handle carry
endmodule
```

```
module tb;
    parameter WIDTH = 16;  // Parameterized width
    reg [WIDTH-1:0] a, b;
    wire [WIDTH-1:0] sum;
    wire carry_out;

    // Instantiate the adder module with parameter WIDTH
    adder #(.WIDTH(WIDTH)) add_instance (
        .a(a),
        .b(b),
        .sum(sum),
        .carry_out(carry_out)
    );

    initial begin
        // samples inputs are assigned in hex notation
        a = 16'h0001; b = 16'h0001; #10;

        a = 16'hFFFF; b = 16'h0001; #10;

        a = 16'h8000; b = 16'h8000; #10;

        a = 16'h1234; b = 16'h4321; #10;

    end
endmodule
```

## Tasks:

1. Using the given design, write a Verilog code at all level of abstraction to make a Module for 1 bit ALU having inputs (1 bit) as A and B as input data, carry_in, select line (2 bit input) to select between different operations i.e. AND, OR and ADD/SUB. The module will have an Output for Result of operations and another output for carry_out. Selection between ADD/SUB will be provided by another input b_inv. Show simulation results for every instruction executed. Make sure to design a circuit diagram using logisim and a truth table to verify the working of ALU.

2. Write Verilog code to build a 4bit ALU out of 4 1bit ALUs. The ALU must be capable of performing ADD, SUB operations and logical operation that are AND and OR operations and a SLT operation. Status flags (CF, SF, ZF and OF) must also be set accordingly Test the code using test bench and simulation.

3. Write a Verilog code for a parameterized ALU, where the parameter N specifies the bit-width of the input and output signals. The operation of the ALU is determined by a separate control signal.
   The ALU should support the following operations:
   a. Addition
   b. Subtraction
   c. Basic logical operations (AND, OR, NOT, XOR)

   The inputs and outputs of the ALU must be N-bit wide, making the design scalable and reusable for different bit sizes.

4. Design and model a fully functional 32-bit ALU using Verilog. The ALU must perform a specified set of arithmetic, logical, and shifting operations (given in table 1) based on a set of control signals, and correctly manage status flags (given in table 2).

| alu_op | Operation | Result | Used by |
|---|---|---|---|
| 0000 | ADD | A + B | add, addi, lw, sw, auipc |
| 0001 | SUB | A − B | sub, beq, bne, blt, bge |
| 0010 | AND | A & B | and, andi |
| 0011 | OR | A \| B | or, ori |
| 0100 | XOR | A ^ B | xor, xori |
| 0101 | SLT | A < B (signed) ? 1 : 0 | slt, slti |
| 0110 | SLTU | A < B (unsigned) ? 1 : 0 | sltu, sltiu |
| 0111 | SLL | A << B[4:0] | sll, slli |
| 1000 | SRL | A >> B[4:0] (logical) | srl, srli |
| 1001 | SRA | A >>> B[4:0] (arithmetic) | sra, srai |

*Table 1: ALU Operation encoding*

| Port | Dir | Width | Meaning |
|---|---|---|---|
| a | input | 32 | operand A, from register file rs1 |
| b | input | 32 | operand B, from rs2 or immediate (selected by ALUSrc mux) |
| alu_op | input | 4 | operation select, from control unit |
| result | output | 32 | computed result, goes to DMEM address or write-back |
| zero | output | 1 | 1 when result==0, used by branch unit for BEQ/BNE |

*Table 2: Port table*

5. Simulate the above (designed in task 4) ALU in Logisim.